

Onway Technical Assessment (Hono.js)

Onway Project — Technical Assessment Task

Candidate: Balen Ibrahim Star **Role:** Full-Stack Software Engineer **Estimated Time:** 2–3 days (take-home)

Purpose

This task evaluates your ability to design, build, and deploy a small end-to-end feature using **Onway’s actual tech stack**, so we can see how you work within our real structure — not a generic setup.

Required Stack (must be used, no substitutions)

- **Backend Framework:** Hono.js (required — no Laravel, no Spring Boot, no plain Express)
- **Runtime:** Node.js or Bun (your choice)
- **Frontend:** React.js or Vue.js
- **Database:** MySQL, PostgreSQL, or MongoDB (your choice — explain your pick)
- **Deployment:** Must be deployed and reachable over HTTPS (VPS, Railway, Render, Cloudflare, etc.)

Note: Laravel and Spring Boot submissions will not be accepted for this assessment, regardless of quality — we need to see Hono.js specifically since it’s part of our current backend structure.

Scenario

Onway needs a lightweight **“Package Delivery Tracker”** module — a simplified version of the delivery workflow used across our platforms (pickup → in transit → out for delivery → delivered).

You will build a small full-stack slice of this feature using the required stack above.

Task Requirements

1. Backend — Hono.js

Build a RESTful API with the following endpoints: - `POST /orders` — create a new delivery order (fields: customer name, pickup address, drop-off address, package weight, status) - `GET /orders` — list all orders (support pagination + filter by status) - `GET /orders/:id` — get single order with full status history - `PATCH /orders/:id/status` — update order status (must log a timestamped status history entry, not just overwrite) - `DELETE /orders/:id` — cancel an order (soft delete, not hard delete)

Requirements: - Use Hono’s routing, middleware, and validation patterns (e.g. `zValidator` with Zod, or Hono’s built-in validator middleware) — no manual `if` chains for validation - Proper error handling via Hono’s error handler (`app.onError()`), returning meaningful HTTP status codes - Status must follow a fixed state machine: `pending` → `picked_up` → `in_transit` → `out_for_delivery` → `delivered` (also allow `cancelled` from any state before `delivered`) - Include a status history sub-schema/table (`order_id`, `status`, `timestamp`)

2. Database

- Use MySQL, PostgreSQL, or MongoDB (your choice)

- Provide schema/migration files (if using an ORM/query builder like Drizzle or Prisma, include the schema file)
- Briefly explain (in README) why you chose relational vs non-relational for this use case

3. Frontend (React.js or Vue.js)

Build a simple dashboard that: - Lists all orders in a table with status badges (color-coded per status) - Allows filtering by status - Has a form to create a new order - Has a detail view showing the status history timeline for a single order - Handles loading states and API errors gracefully

4. Deployment

- Deploy the working app (frontend + backend + DB) with HTTPS enabled — Hono works well on Node, Bun, or edge runtimes (Cloudflare Workers), pick whichever fits your setup
- Provide the live link + GitHub repo link

5. Documentation

Include a short `README.md` with: - Setup/run instructions - Why Hono.js patterns were used the way they were (middleware, validation, routing structure) - Architecture decisions and tradeoffs - What you'd improve with more time

Bonus (Optional — not required, but a strong signal)

- Add JWT authentication using a Hono middleware so only logged-in staff can update order status
- Add a simple React Native or mobile-responsive view showing the courier's assigned orders
- Add unit tests for the status transition logic (Hono supports easy testing via `app.request()`)

What We're Evaluating

Area	What we're looking for
Hono.js usage	Correct use of routing, middleware, validation — not just Express habits copy-pasted in
Data modeling	Sensible schema, correct handling of state history
Frontend quality	Clean component structure, responsive UI, good UX for status states
Code quality	Readability, structure, naming, separation of concerns
Deployment	Can they ship something live and working on our actual stack
Communication	Clarity of README, explanation of decisions

Submission

Please submit: 1. GitHub repo link (public or invite access) 2. Live deployed link 3. README with setup + notes

Reply directly to this task with your submission once ready.